

SUBMITTED BY:

NAME: MARIHA MALIK
ROLLNO: 23-BCS-060
SECTION: A
SESSION: 2023-2027

SUBMITTED TO:

INSTRUCTOR: RESPECTED Mr. WAQAS SALEEM
COURSE TITLE: DEVOPS

SUBMISSION DATE:

OCTOBER 18, 2026

Table of Contents

Lab Overview and Objectives 3
Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase) 3
Task 2 – Creating and Resolving Merge Conflicts Manually 6
Task 3 – Managing Ignored Files with .gitignore and Removing Tracked Files 9
Task 4 – Create Temporary Changes and Use git stash 12
Task 5 – Checkout a Specific Commit Using git log 13
Task 6 – Resetting Commits (Soft vs Hard Reset) 15
Task 7 – Amending the Last Commit 18
Task 8 – Reverting a Commit (Safe Undo on Remote Branch) 19
Task 9 – Force Push 20
Task 10 – Running Gitea in GitHub Codespaces via Docker Compose 22
Task 11 – Creating a GitHub Pages Portfolio Site 26
Summary 28

Lab Overview and Objectives

This laboratory work exercises advanced version control workflows that help manage commit history, recover previous versions, and safely undo mistakes both locally and remotely in an Ubuntu/VirtualBox environment. All operations were verified under the GitHub identity mareeh58.

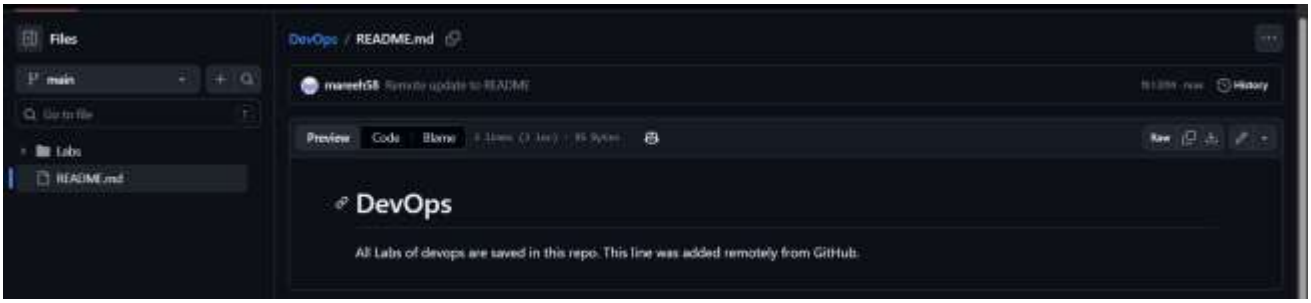
Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase)

Objective and Details: Demonstrates what happens when your local repository and remote repository both have new commits — and how git pull and git pull --rebase behave differently.

Commands Executed:

```
# 1. Edit README.md on GitHub and commit
# 2. Edit README.md locally and commit:
git add README.md
git commit -m "Local update to README"
# 3. Try to push (rejected):
git push origin main
# 4. Pull using merge:
git pull --no-rebase origin main
git push -u origin main
# 5. Pull using rebase:
git pull --rebase origin main
git push -u origin main
```

Verification Evidence (Screenshots):



remote_edit

```
mareeh58@ubuntu:~/DevOps$ git add README.md
mareeh58@ubuntu:~/DevOps$ git commit -m "LOcal update to README"
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
mareeh58@ubuntu:~/DevOps$
```

local_commit

```
mareeh58@ubuntu:~/DevOps$ git push origin main
To github.com:mareeh58/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:mareeh58/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
mareeh58@ubuntu:~/DevOps$
```

push_error

```
mareeh58@ubuntu:~/DevOps$ git pull --no-rebase origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1008 bytes | 252.00 KiB/s, done.
From github.com:mareeh58/DevOps
 * branch            main       -> FETCH_HEAD
   4eb517d..fb130f4  main       -> origin/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
mareeh58@ubuntu:~/DevOps$
```

merge_commit

```
mareeh58@ubuntu:~/DevOps$ git push -u origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 624 bytes | 312.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To github.com:mareeh58/DevOps.git
   fb130f4..90ba4b0  main -> main
branch 'main' set up to track 'origin/main'.
mareeh58@ubuntu:~/DevOps$
```

push_after_merge

```
mareeh58@ubuntu:~/DevOps$ git pull --rebase origin main
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 1.86 KiB | 238.00 KiB/s, done.
From github.com:mareeh58/DevOps
 * branch            main       -> FETCH_HEAD
   9299ef0..e44a9b2  main       -> origin/main
Updating 9299ef0..e44a9b2
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
mareeh58@ubuntu:~/DevOps$
```

rebase_pull

```
mareeh58@ubuntu:~/DevOps$ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
mareeh58@ubuntu:~/DevOps$
```

push_after_rebase

Task 2 – Creating and Resolving Merge Conflicts Manually

Objective and Details: Understanding how Git handles file conflicts when two people modify the same line of code differently.

Commands Executed:

```
# 1. Edit conflicting line on GitHub and commit
# 2. Edit same line locally and commit:
git add README.md
git commit -m "Local conflicting change"

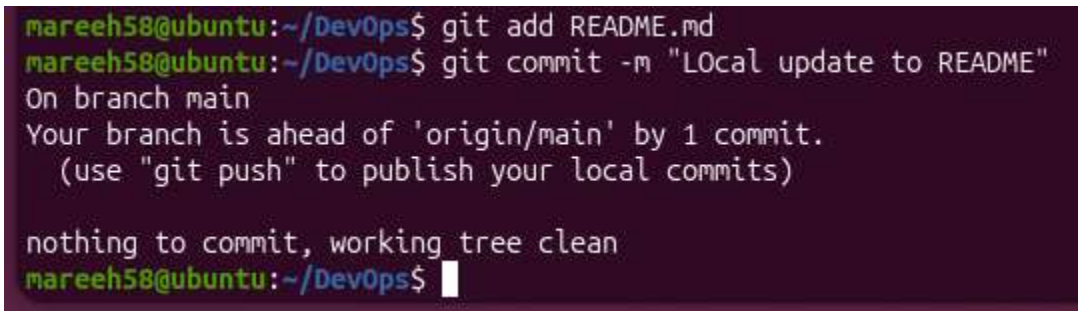
# 3. Push and trigger conflict:
git push -u origin main
git pull --rebase origin main

# 4. Resolve conflict in README.md, then stage & continue rebase:
git add README.md
git rebase --continue
git push -u origin main
```

Verification Evidence (Screenshots):



remote_conflict_edit



local_conflict_edit

```
mareeh58@ubuntu:~/DevOps$ git add README.md
mareeh58@ubuntu:~/DevOps$ git commit -m "Local conflicting change"
[main 4258d20] Local conflicting change
 1 file changed, 1 insertion(+), 1 deletion(-)
mareeh58@ubuntu:~/DevOps$
```

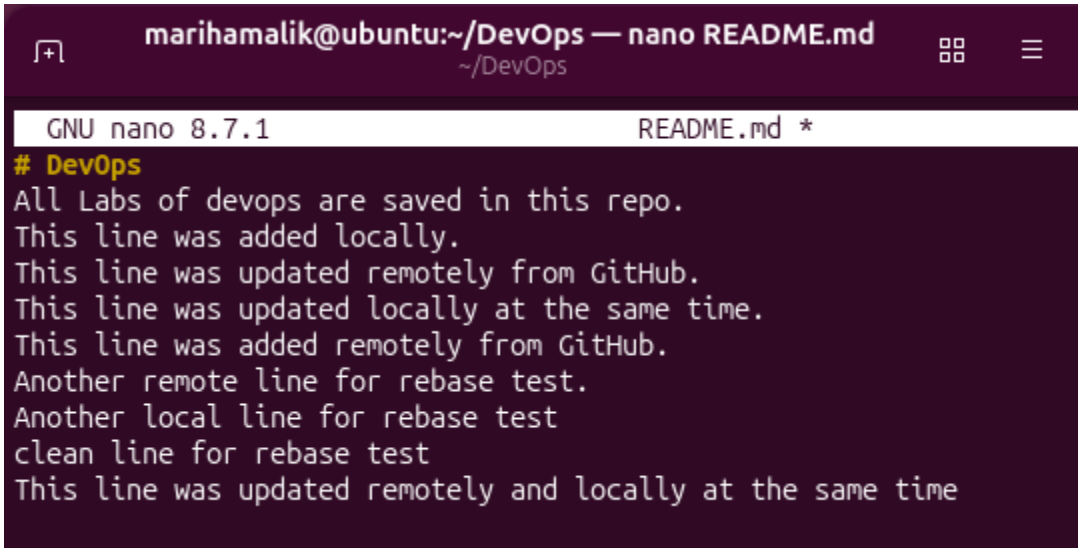
local_conflict_commit

```
mareeh58@ubuntu:~/DevOps$ git push -u origin main
To github.com:mareeh58/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:mareeh58/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
mareeh58@ubuntu:~/DevOps$
```

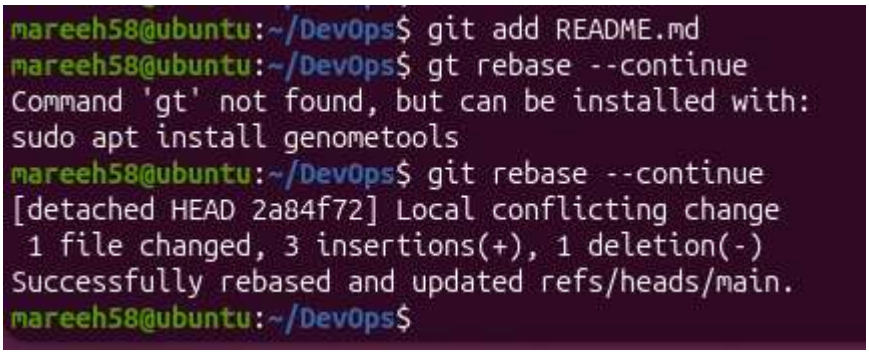
conflict_push_error

```
mareeh58@ubuntu:~/DevOps$ git pull --rebase origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 957 bytes | 319.00 KiB/s, done.
From github.com:mareeh58/DevOps
 * branch            main       -> FETCH_HEAD
   e44a9b2..cd8f3a9  main       -> origin/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
error: could not apply 4258d20... Local conflicting change
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --
abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Could not apply 4258d20... # Local conflicting change
mareeh58@ubuntu:~/DevOps$
```

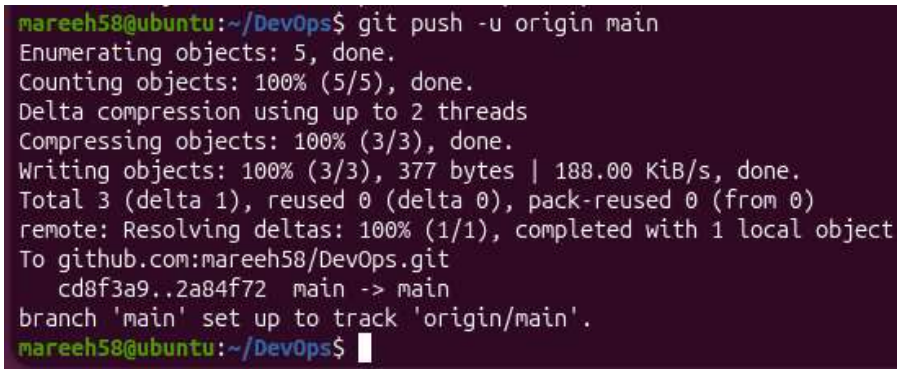
conflict_message



resolved_readme



rebase_continue



push_after_resolve

Task 3 – Managing Ignored Files with .gitignore and Removing Tracked Files

Objective and Details: Using a .gitignore file to exclude files or directories from version control and removing them from Git's tracking while keeping them locally.

Commands Executed:

```
mkdir textfiles
echo "File A content" > textfiles/a.txt
git add .
git commit -m "Added textfiles directory with three files"
echo "textfiles/*" > .gitignore
git add .gitignore
git commit -m "Added .gitignore to ignore textfiles directory"
git rm -r --cached textfiles
git commit -m "Removed tracked textfiles directory"
git push origin main
```

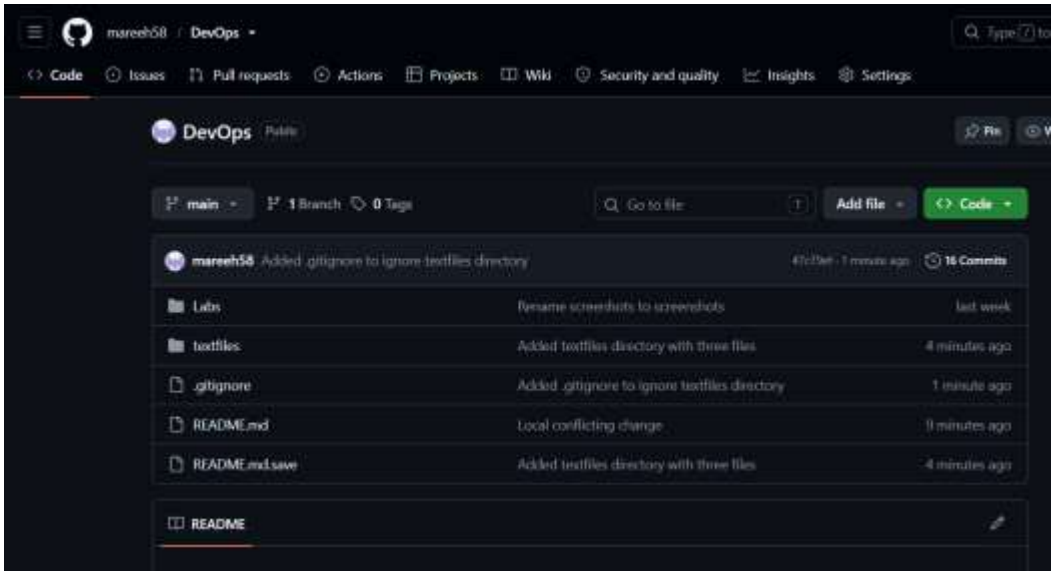
Verification Evidence (Screenshots):

```
mareeh58@ubuntu:~/DevOps$ cd ~/DevOps
mareeh58@ubuntu:~/DevOps$ mkdir textfiles
mareeh58@ubuntu:~/DevOps$ echo "File A content" > textfiles/a.txt
mareeh58@ubuntu:~/DevOps$ echo "File B content" > textfiles/b.txt
mareeh58@ubuntu:~/DevOps$ echo "File C content" > textfiles/c.txt
mareeh58@ubuntu:~/DevOps$ git add .
mareeh58@ubuntu:~/DevOps$ git commit -m "Added textfiles directory with three files"
[main a08a2e6] Added textfiles directory with three files
 4 files changed, 9 insertions(+)
 create mode 100644 README.md.save
 create mode 100644 textfiles/a.txt
 create mode 100644 textfiles/b.txt
 create mode 100644 textfiles/c.txt
mareeh58@ubuntu:~/DevOps$ git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 667 bytes | 83.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:mareeh58/DevOps.git
   2a84f72..a08a2e6  main -> main
mareeh58@ubuntu:~/DevOps$
```

push_textfiles

```
mareeh58@ubuntu:~/DevOps$ echo "textfiles/*" > .gitignore
mareeh58@ubuntu:~/DevOps$ gt add .gitignore
Command 'gt' not found, but can be installed with:
sudo apt install genometools
mareeh58@ubuntu:~/DevOps$ git add .gitignore
mareeh58@ubuntu:~/DevOps$ git commit -m "Added .gitignore to ignore textfiles directory"
[main 47c79ef] Added .gitignore to ignore textfiles directory
 1 file changed, 1 insertion(+)
 create mode 100644 .gitignore
mareeh58@ubuntu:~/DevOps$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 308 bytes | 154.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:mareeh58/DevOps.git
   a08a2e6..47c79ef  main -> main
mareeh58@ubuntu:~/DevOps$
```

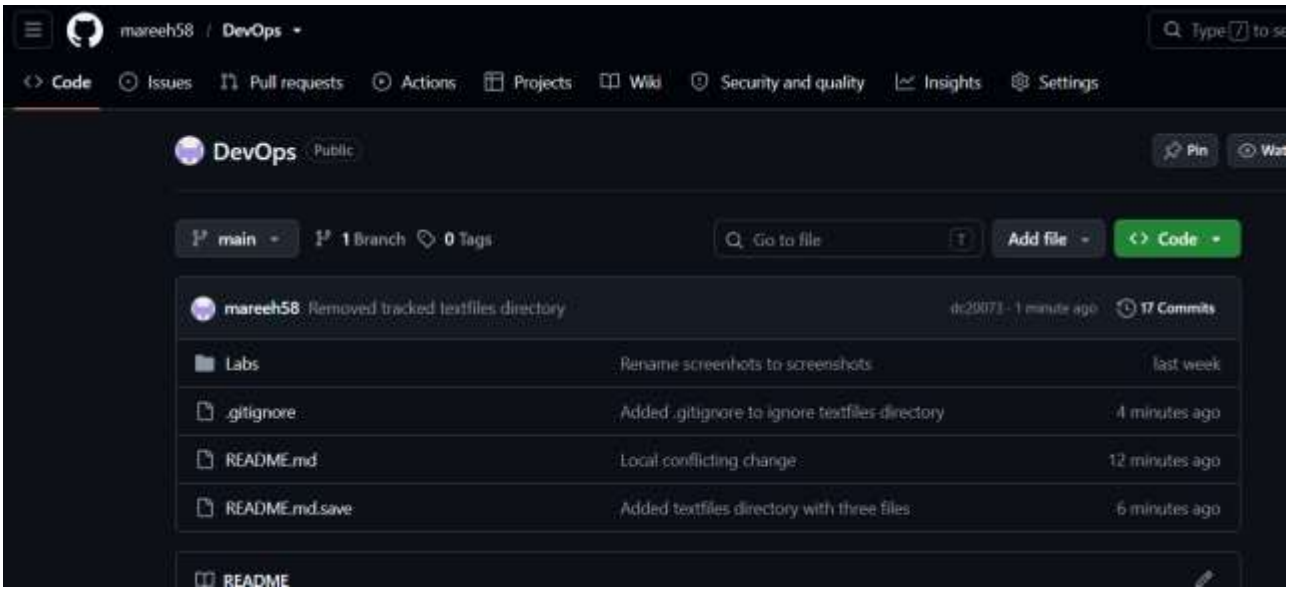
gitignore_push



repo_still_has_textfiles

```
mareeh58@ubuntu:~/DevOps$ git rm -r --cached textfiles
rm 'textfiles/a.txt'
rm 'textfiles/b.txt'
rm 'textfiles/c.txt'
mareeh58@ubuntu:~/DevOps$ git add .
mareeh58@ubuntu:~/DevOps$ git commit -m "Removed tracked textfiles directory"
[main dc20073] Removed tracked textfiles directory
 3 files changed, 3 deletions(-)
 delete mode 100644 textfiles/a.txt
 delete mode 100644 textfiles/b.txt
 delete mode 100644 textfiles/c.txt
mareeh58@ubuntu:~/DevOps$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 240 bytes | 240.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:mareeh58/DevOps.git
   47c79ef..dc20073  main -> main
mareeh58@ubuntu:~/DevOps$
```

rm_cached_push



repo_textfiles_removed

Task 4 – Create Temporary Changes and Use git stash

Objective and Details: Demonstrates how git stash allows you to temporarily save uncommitted changes so that you can safely switch branches without losing work.

Commands Executed:

```
git checkout -b feature-branch
# Modify README.md
git checkout main  # Error: changes would be overwritten
git stash
git checkout main
git checkout feature-branch
git status
git stash pop
```

Verification Evidence (Screenshots):

```
mareeh58@ubuntu:~/DevOps$ git checkout -b feature-branch
Switched to a new branch 'feature-branch'
mareeh58@ubuntu:~/DevOps$ echo "Intial test line on feature-branch" >> README.md
mareeh58@ubuntu:~/DevOps$ git add README.md
mareeh58@ubuntu:~/DevOps$ git commit -m "changes the README.md file"
[feature-branch 72293ac] changes the README.md file
 1 file changed, 1 insertion(+)
mareeh58@ubuntu:~/DevOps$
```

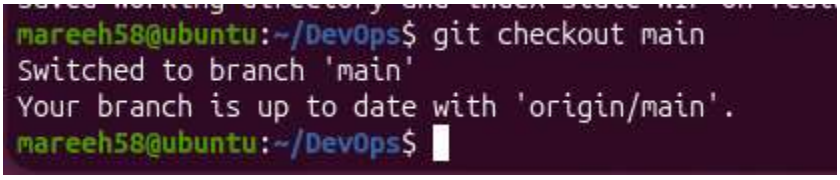
modified_readme

```
mareeh58@ubuntu:~/DevOps$ echo "Temporary unstashed line" >> README.md
mareeh58@ubuntu:~/DevOps$ git checkout main
error: Your local changes to the following files would be overwritten by checkout:
        README.md
Please commit your changes or stash them before you switch branches.
Aborting
mareeh58@ubuntu:~/DevOps$
```

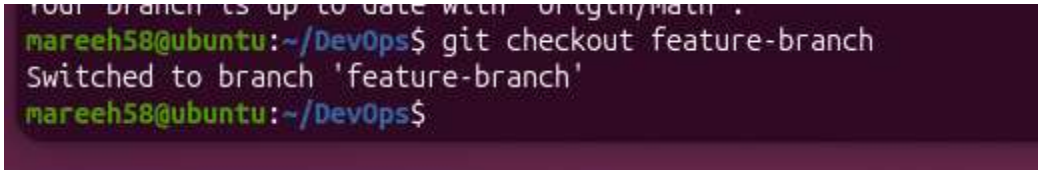
checkout_error

```
mareeh58@ubuntu:~/DevOps$ git stash
Saved working directory and index state WIP on feature-branch: 72293ac changes the README.md file
mareeh58@ubuntu:~/DevOps$
```

stash_command



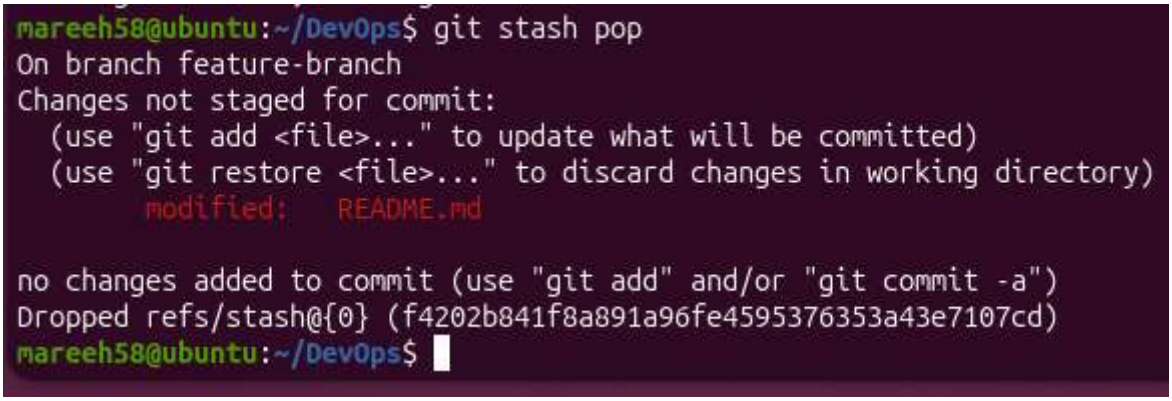
branch_switched



back_to_feature



status_clean



stash_pop

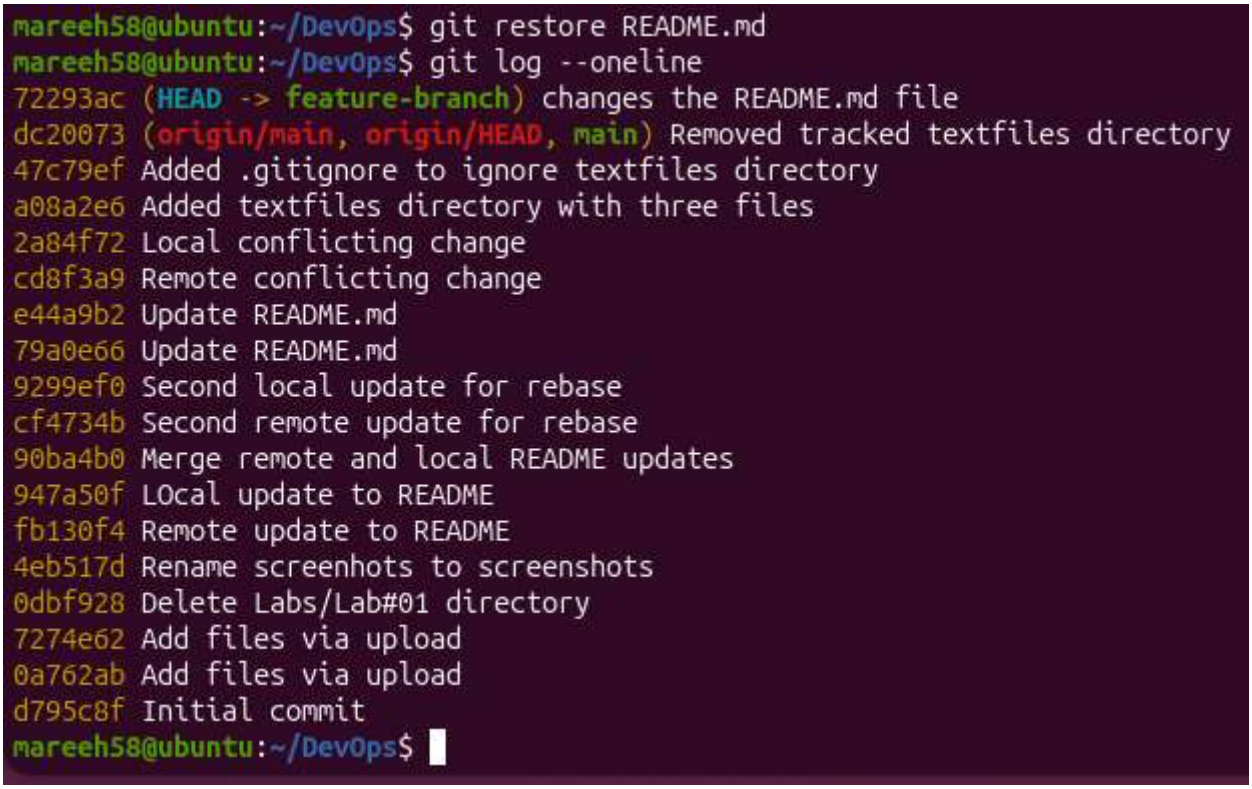
Task 5 – Checkout a Specific Commit Using git log

Objective and Details: Revisit past versions by checking out a specific commit hash and operating in detached HEAD state.

Commands Executed:

```
git log --oneline
git checkout <commit-hash>
git checkout main
```

Verification Evidence (Screenshots):



log_before_checkout

```
mareeh58@ubuntu:~/DevOps$ git checkout 2a84f722231dd76a16b18dfe769049e16835345f
Note: switching to '2a84f722231dd76a16b18dfe769049e16835345f'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 2a84f72 Local conflicting change
mareeh58@ubuntu:~/DevOps$
```

detached_head

```
mareeh58@ubuntu:~/DevOps$ git checkout main
Previous HEAD position was 2a84f72 Local conflicting change
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
mareeh58@ubuntu:~/DevOps$
```

back_to_main

Task 6 – Resetting Commits (Soft vs Hard Reset)

Objective and Details: Demonstrates the difference between a soft and hard reset in Git with step-by-step file and history verification.

Commands Executed:

```
git add .
git commit -m "Added test line"
# Add second commit
git log --oneline
git reset --soft HEAD~1
git reset --hard HEAD~1
```

Verification Evidence (Screenshots):

```
mareeh58@ubuntu:~/DevOps$ echo "Line for first commit in Task 6" >> README.md
mareeh58@ubuntu:~/DevOps$ git add .
mareeh58@ubuntu:~/DevOps$ git commit -m "Added test line"
[main d99ef0d] Added test line
 1 file changed, 1 insertion(+)
mareeh58@ubuntu:~/DevOps$
```

first_commit

```
mareeh58@ubuntu:~/DevOps$ echo "Line for second commit in Task 6" >> README.md
mareeh58@ubuntu:~/DevOps$ git add .
mareeh58@ubuntu:~/DevOps$ git commit -m "Second test line"
[main 82db5f1] Second test line
 1 file changed, 1 insertion(+)
mareeh58@ubuntu:~/DevOps$
```

second_commit

```
mareeh58@ubuntu:~/DevOps$ git log -2 --oneline
82db5f1 (HEAD -> main) Second test line
d99ef0d Added test line
mareeh58@ubuntu:~/DevOps$
```

log_before_reset

```
mareeh58@ubuntu:~/DevOps$ cat README.md
# DevOps
All Labs of devops are saved in this repo.
This line was added locally.
This line was updated remotely from GitHub.
This line was updated locally at the same time.
This line was added remotely from GitHub.
Another remote line for rebase test.
Another local line for rebase test
clean line for rebase test
This line was updated remotely and locally at the same time
Line for first commit in Task 6
Line for second commit in Task 6
mareeh58@ubuntu:~/DevOps$
```

file_before_reset

```
mareeh58@ubuntu:~/DevOps$ git reset --soft HEAD~1
mareeh58@ubuntu:~/DevOps$
```

soft_reset

```
mareeh58@ubuntu:~/DevOps$ git log -2 --oneline
d99ef0d (HEAD -> main) Added test line
dc20073 (origin/main, origin/HEAD) Removed tracked textfiles directory
mareeh58@ubuntu:~/DevOps$
```

log_after_soft_reset

```
mareeh58@ubuntu:~/DevOps$ cat README.md
# DevOps
All Labs of devops are saved in this repo.
This line was added locally.
This line was updated remotely from GitHub.
This line was updated locally at the same time.
This line was added remotely from GitHub.
Another remote line for rebase test.
Another local line for rebase test
clean line for rebase test
This line was updated remotely and locally at the same time
Line for first commit in Task 6
Line for second commit in Task 6
mareeh58@ubuntu:~/DevOps$
```

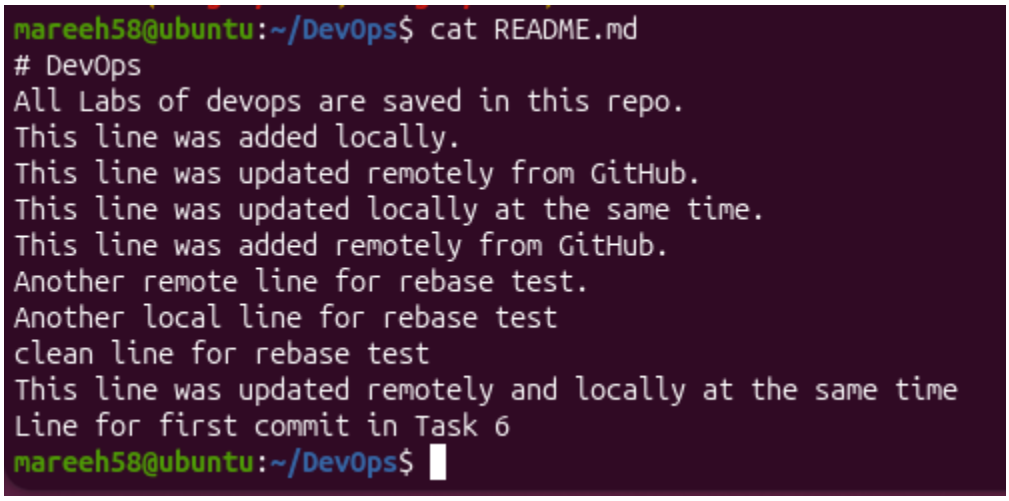
file_after_soft_reset

```
mareeh58@ubuntu:~/DevOps$ git reset --hard HEAD~1
HEAD is now at d99ef0d Added test line
mareeh58@ubuntu:~/DevOps$
```

hard_reset

```
mareeh58@ubuntu:~/DevOps$ git log -2 --oneline
d99ef0d (HEAD -> main) Added test line
dc20073 (origin/main, origin/HEAD) Removed tracked textfiles directory
mareeh58@ubuntu:~/DevOps$
```

log_after_hard_reset



file_after_hard_reset

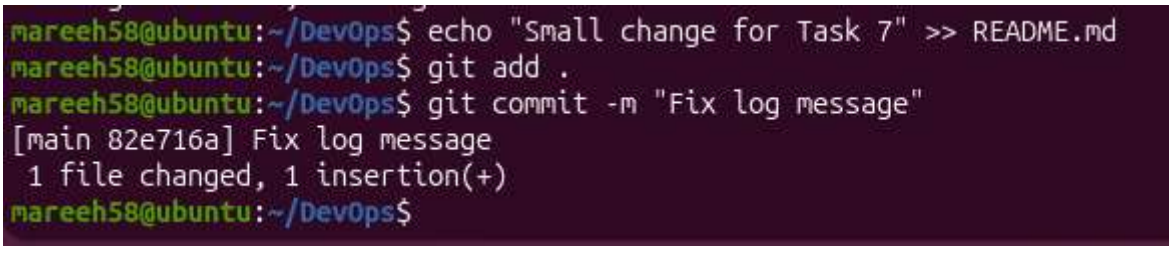
Task 7 – Amending the Last Commit

Objective and Details: Modifying your last commit, including its message or staged files using git commit --amend.

Commands Executed:

```
git add .
git commit -m "Fix log message"
# Update another file
git add .
git commit --amend
```

Verification Evidence (Screenshots):



first_amend_commit

```
mareeh58@ubuntu:~/DevOps$ echo "This was the forgotten file update" : > forgotten_file.txt
mareeh58@ubuntu:~/DevOps$ git add .
mareeh58@ubuntu:~/DevOps$ git commit --amend --no-edit
[main 2ad0f0a] Fix log message
 Date: Sat Oct 17 03:25:57 2026 +0000
 2 files changed, 2 insertions(+)
 create mode 100644 forgotten_file.txt
mareeh58@ubuntu:~/DevOps$
```

amend_commit

Task 8 – Reverting a Commit (Safe Undo on Remote Branch)

Objective and Details: Safely undoing published commits by creating a new inverse commit without rewriting shared history.

Commands Executed:

```
echo "temporary text" >> temp.txt
git add .
git commit -m "Added temporary text file"
git push origin main
git log --oneline
git revert <commit-hash>
git push origin main
```

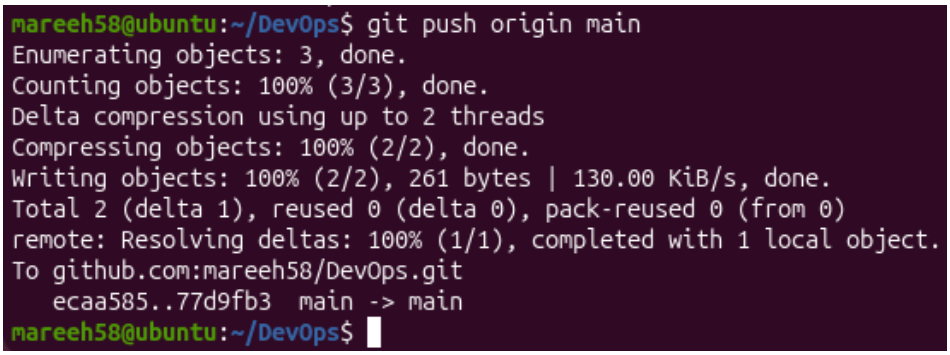
Verification Evidence (Screenshots):

```
mareeh58@ubuntu:~/DevOps$ echo "temporary text" >> temp.txt
mareeh58@ubuntu:~/DevOps$ git add .
mareeh58@ubuntu:~/DevOps$ git commit -m "Added temporary text file"
[main ecaa585] Added temporary text file
 1 file changed, 1 insertion(+)
 create mode 100644 temp.txt
mareeh58@ubuntu:~/DevOps$ git push origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 2 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (10/10), 967 bytes | 483.00 KiB/s, done.
Total 10 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 1 local object.
To github.com:mareeh58/DevOps.git
   dc20073..ecaa585  main -> main
mareeh58@ubuntu:~/DevOps$
```

commit_temp_file



revert_commit



revert_push

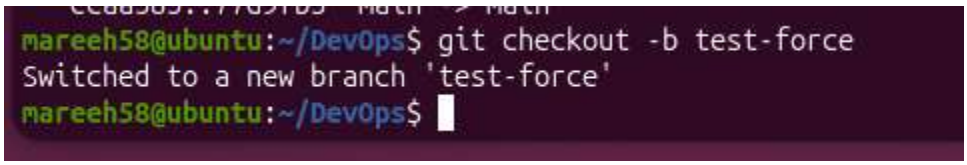
Task 9 – Force Push

Objective and Details: Rewriting remote branch history with git push --force on dedicated branches.

Commands Executed:

```
git checkout -b test-force
# Make commit
git push origin test-force
git reset --hard HEAD~1
git push origin test-force
git push origin test-force --force
```

Verification Evidence (Screenshots):



new_branch

```
mareeh58@ubuntu:~/DevOps$ echo "Testng force push feature" >> README.md
mareeh58@ubuntu:~/DevOps$ git add .
mareeh58@ubuntu:~/DevOps$ git commit -m "Testing force push"
[test-force c52e731] Testing force push
 1 file changed, 1 insertion(+)
mareeh58@ubuntu:~/DevOps$
```

force_commit

```
mareeh58@ubuntu:~/DevOps$ git push origin test-force
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 319 bytes | 319.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'test-force' on GitHub by visiting:
remote:      https://github.com/mareeh58/DevOps/pull/new/test-force
remote:
To github.com:mareeh58/DevOps.git
 * [new branch]      test-force -> test-force
mareeh58@ubuntu:~/DevOps$
```

push_force_branch

```
mareeh58@ubuntu:~/DevOps$ git reset --hard HEAD~1
HEAD is now at 77d9fb3 Revert "Added temporary text file"
mareeh58@ubuntu:~/DevOps$
```

hard_reset_force

```
mareeh58@ubuntu:~/DevOps$ git push origin test-force
To github.com:mareeh58/DevOps.git
 ! [rejected]        test-force -> test-force (non-fast-forward)
error: failed to push some refs to 'github.com:mareeh58/DevOps.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
mareeh58@ubuntu:~/DevOps$
```

normal_push

```
mareeh58@ubuntu:~/DevOps$ git push origin test-force --force
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:mareeh58/DevOps.git
 + c52e731...77d9fb3 test-force -> test-force (forced update)
mareeh58@ubuntu:~/DevOps$
```

force_push

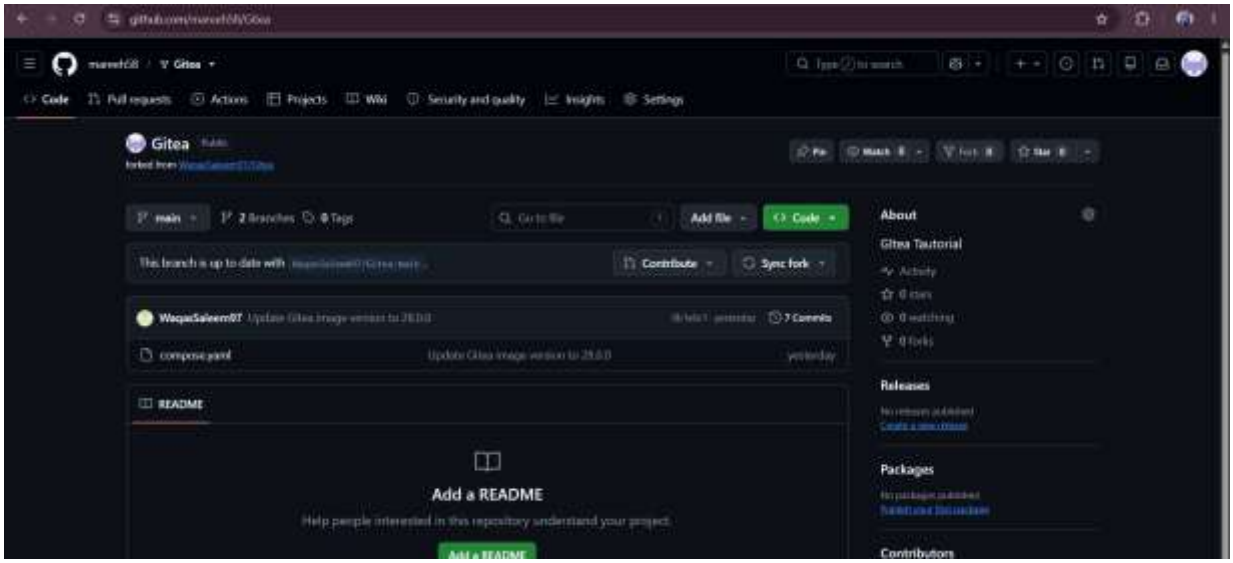
Task 10 – Running Gitea in GitHub Codespaces via Docker Compose

Objective and Details: Forking repository, running full web application in GitHub Codespace via Docker Compose, and interacting with Gitea self-hosted Git service.

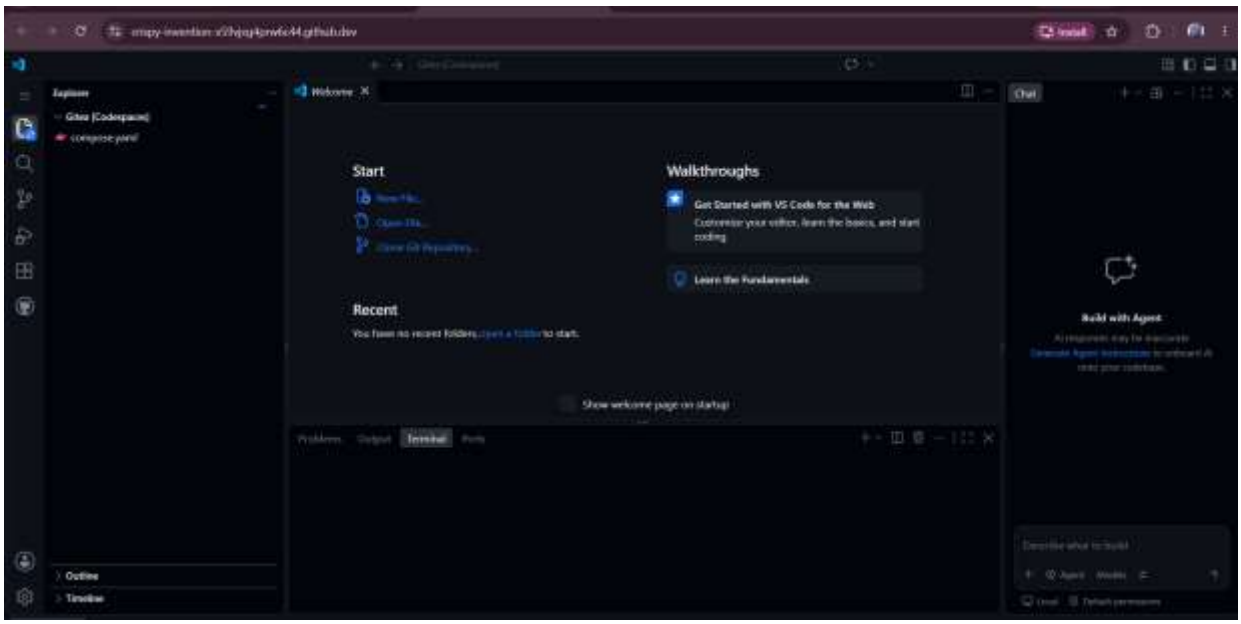
Commands Executed:

```
docker compose up -d
# Configured Gitea web interface on port 3000 (public) and created admin account & lab3-demo repository
```

Verification Evidence (Screenshots):



forked_gitea

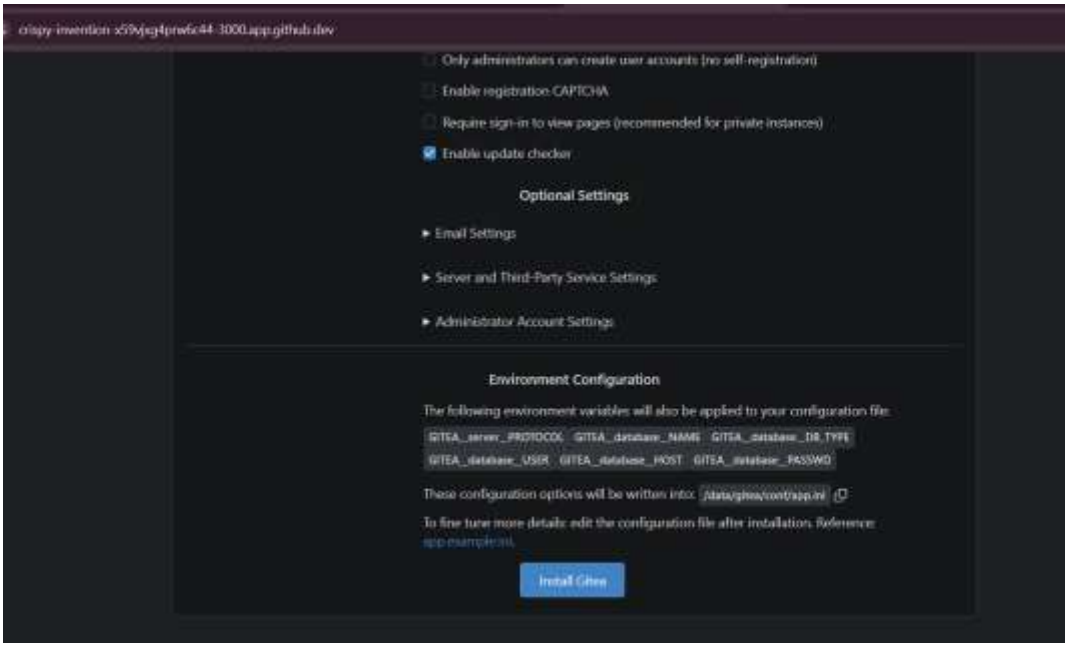


codespace_loading

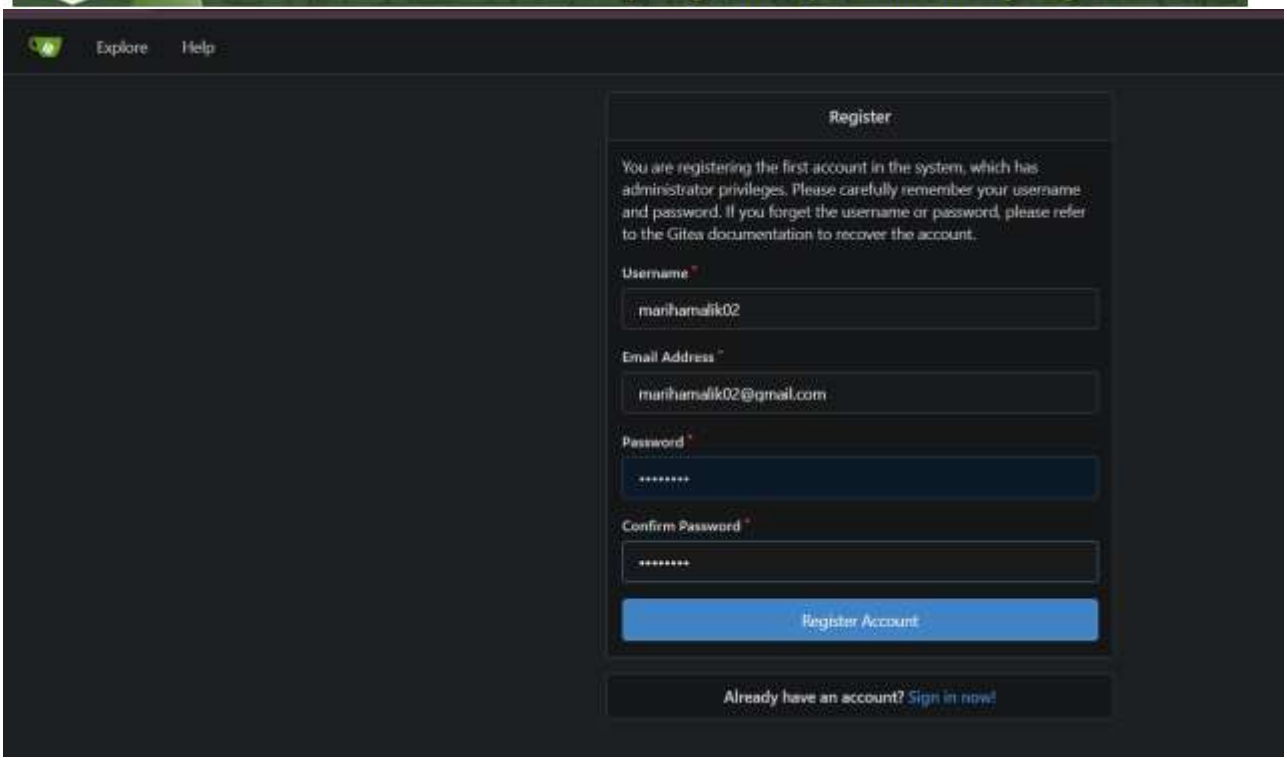
```
Problems  Output  Terminal  Ports

@mareeh58 →/workspaces/Gitea (main) $ git checkout codespace
branch 'codespace' set up to track 'origin/codespace'.
Switched to a new branch 'codespace'
@mareeh58 →/workspaces/Gitea (codespace) $ docker compose up -d
[+] up 25/25
 ✓ Image postgres:17-alpine Pulled
 ✓ Image gitea/gitea:28.0.0 Pulled
 ✓ Network webnet           Created
 ✓ Volume gitea             Created
 ✓ Volume gitea_postgres    Created
 ✓ Container gitea_db       Started
 ✓ Container gitea          Started
@mareeh58 →/workspaces/Gitea (codespace) $
```

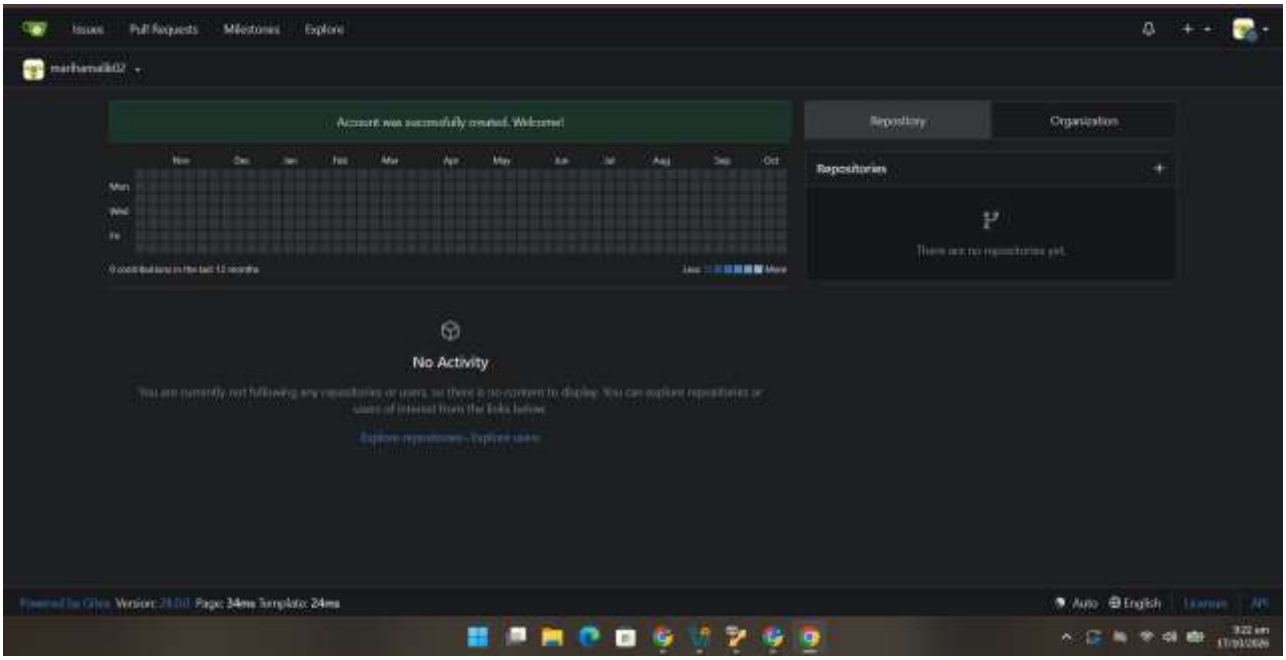
docker_up



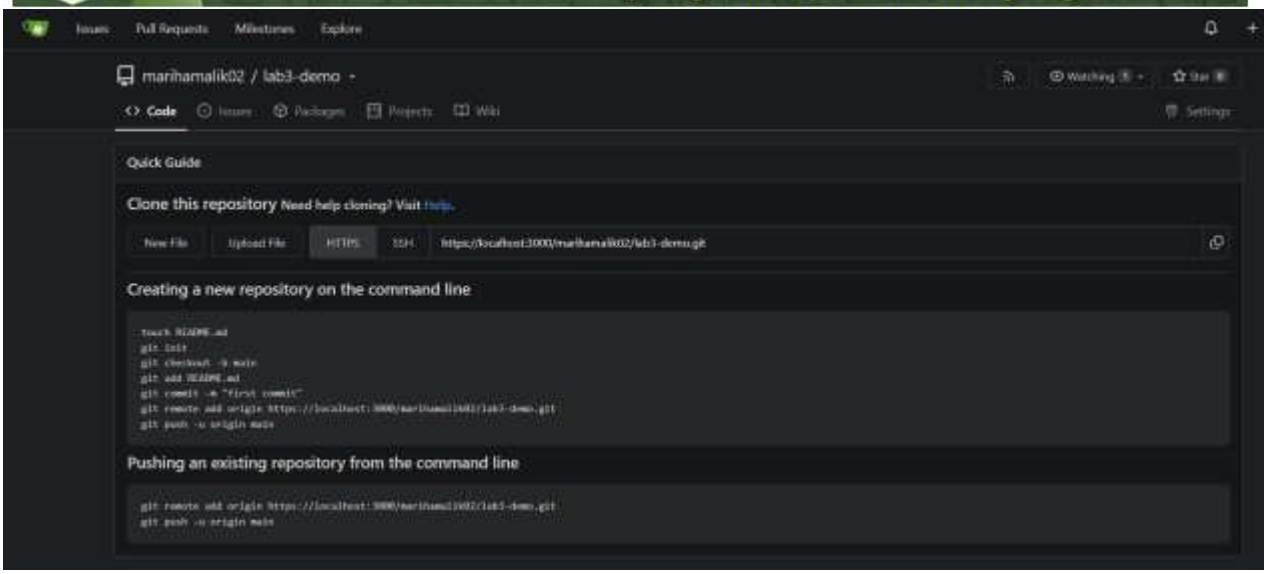
gitea_install_page



admin_setup



gitea_dashboard



gitea_new_repo

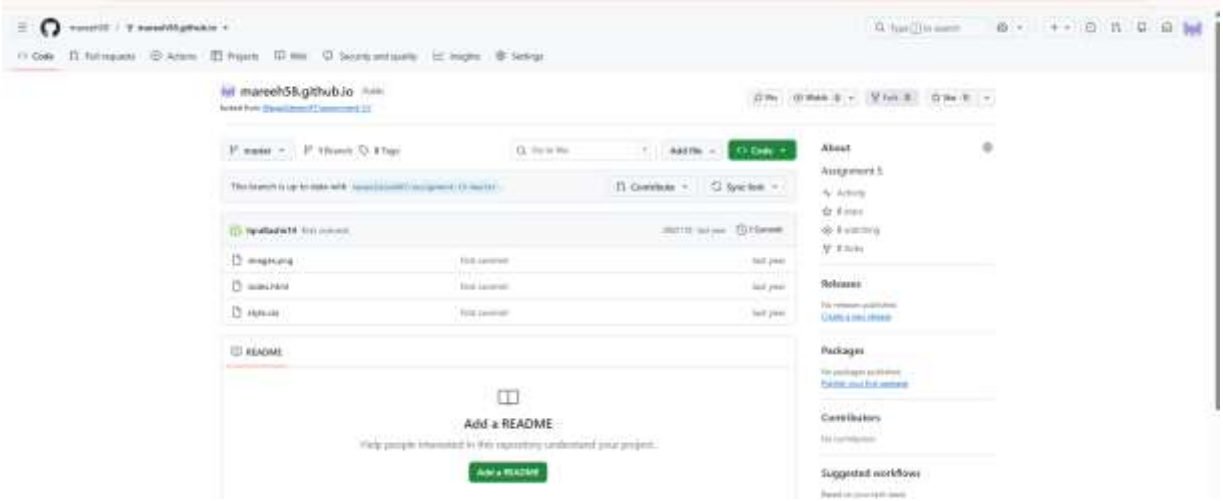
Task 11 – Creating a GitHub Pages Portfolio Site

Objective and Details: Creating a personal portfolio/static site using GitHub Pages by forking teacher CV template, renaming to mareeh58.github.io, updating personal details in index.html for Mariha Malik (23-BCS-060), and pushing via SSH.

Commands Executed:

```
cd ~
rm -rf ~/portfolio
git clone git@github.com:mareeh58/mareeh58.github.io.git ~/portfolio
cd ~/portfolio
nano index.html
git init
git remote add origin git@github.com:mareeh58/mareeh58.github.io.git
git add .
git commit -m "Add portfolio site for GitHub Pages"
git push -u origin main
```

Verification Evidence (Screenshots):



github_pages_repo

```
Username for 'https://github.com': Password for 'https://github.com':
mareeh58@ubuntu:~$ git clone https://github.com/mareeh58/mareeh58.github.io.git ~/portfolio
Cloning into '/home/marihamalik/portfolio'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), 8.00 KiB | 2.67 MiB/s, done.
mareeh58@ubuntu:~$ cd ~/portfolio
mareeh58@ubuntu:~/portfolio$ nano index.html
mareeh58@ubuntu:~/portfolio$ ls -la ~/portfolio
total 32
drwxrwxr-x  3 marihamalik marihamalik 4096 Oct 18 06:02 .
drwxr-x--- 20 marihamalik marihamalik 4096 Oct 18 05:59 ..
drwxrwxr-x  7 marihamalik marihamalik 4096 Oct 18 05:59 .git
-rw-rw-r--  1 marihamalik marihamalik 5465 Oct 18 05:59 images.png
-rw-rw-r--  1 marihamalik marihamalik 4818 Oct 18 06:02 index.html
-rw-rw-r--  1 marihamalik marihamalik 1779 Oct 18 05:59 style.css
mareeh58@ubuntu:~/portfolio$
```

local_static_site

```
mareeh58@ubuntu:~/portfolio$ git remote set-url origin git@github.com:mareeh58/mareeh58.github.io.git
mareeh58@ubuntu:~/portfolio$ git push origin master
Everything up-to-date
mareeh58@ubuntu:~/portfolio$
```

push_static_site



github_pages_settings



live_site

Summary

In this lab, you practiced a broad range of advanced Git commands and real-world development workflows including conflict resolution, manual merges, .gitignore exclusions, stashing, soft/hard resets, commit amending, reverting, force pushing, Gitea Docker deployment, and publishing personal static portfolios live on GitHub Pages at https://mareeh58.github.io/.